

MICROSERVICES COURSE

Session 2



API Gateway & Routing

Single Entry Point · Route Predicates · Filters · Load Balancing



Dr. ElSayed Mohamed Elsayed Baladoh

Tech Lead · Ph.D. · sayedbaladoh@yahoo.com

Monday

3:00 PM – 5:30 PM

Online Session

2.5 Hours

Phase 1

Session 2 of 29

Phase 1 — Foundation & Core Patterns



You are here — Session 2 of 8 in Phase 1. Last session we built the foundation. Today we add the single entry point.

What You Already Have Running



Config Server

:8888

Centralized configuration for all services



Eureka Server

:8761

Service discovery — services find each other by name



Product Service

:8081

REST CRUD API — registered with Eureka + Config

❓ Problem: clients still call `http://localhost:8081/api/products` directly — they must know the exact address and port of every service.

What Happens Without a Gateway?

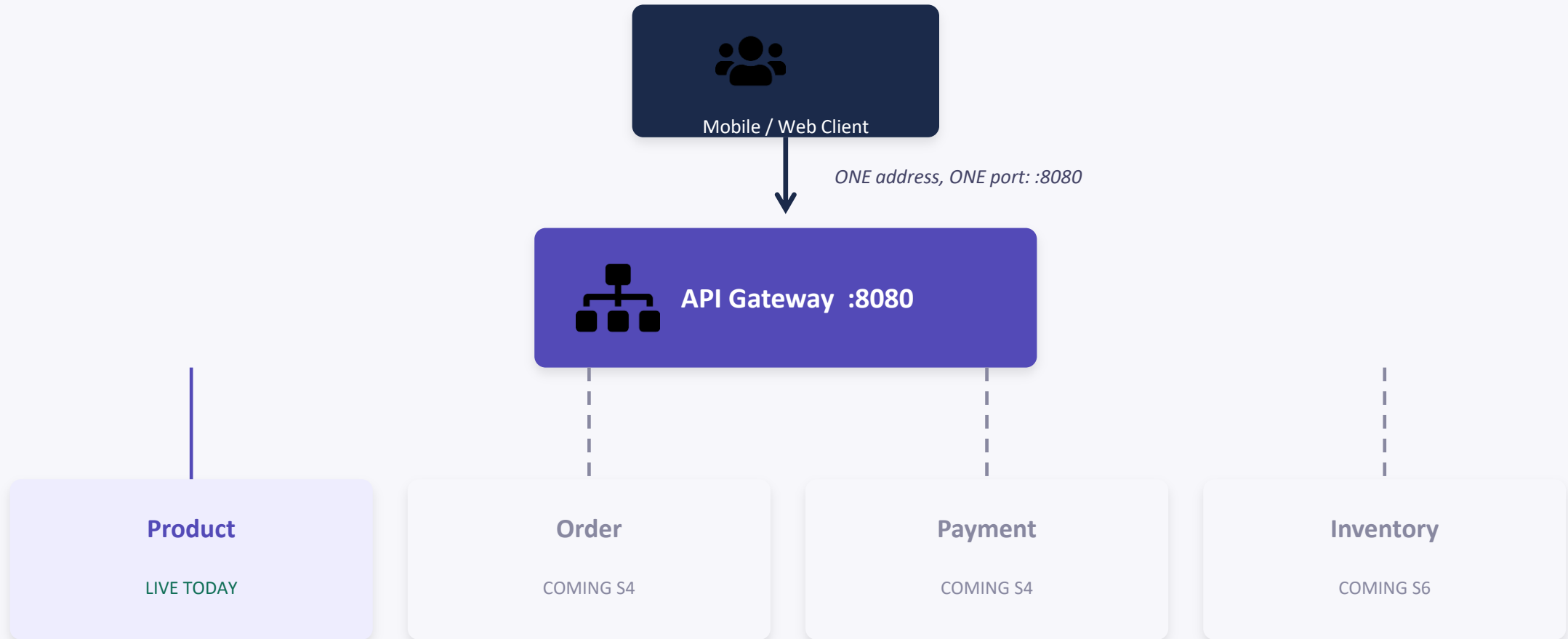
Mobile App _____

```
└→ http://product-service:8081/api/products    ← hardcoded!
└→ http://order-service:8082/api/orders        ← hardcoded!
└→ http://payment-service:8083/api/payments    ← hardcoded!
└→ http://notification-service:8084/notify     ← hardcoded!
```

- ✗ Client must know ALL service addresses
- ✗ No single place to add authentication
- ✗ No single place to add rate limiting
- ✗ Scaling changes the port — must update ALL clients

🔗 *If we add Inventory Service tomorrow, how many places do we need to update?*

API Gateway — The Single Entry Point



Clients only ever know about the Gateway. Everything behind it is invisible — and can change freely.

Six Responsibilities — One Slide



Routing

Match request → forward to right service

✓ TODAY



Load Balancing

Distribute traffic across instances

✓ TODAY



Authentication

Validate JWT ONCE — services trust the Gateway

📖 SESSION 3+



Rate Limiting

Protect services — 100 req/min per client

📖 SESSION 3+



Observability

Log, trace, monitor all traffic in one place

📖 SESSION 3+



Request Shaping

Add / remove / modify headers before forwarding

✓ TODAY

How a Request Flows Through the Gateway

1

Client sends GET /api/products

2

Gateway receives request on port 8080

3

Route Predicates evaluated — does path match any route?

4

YES → apply pre-filters (log request, add request-id)

5

Gateway asks Eureka: "Where is PRODUCT-SERVICE?"

6

Eureka returns:
http://192.168.1.10:8081

7

Gateway forwards the request to that address

8

Response comes back → apply post-filters (add headers)

9

Gateway returns response to client

Route Predicates — 5 Types

Predicate	Example	Use Case
Path	Path=/api/products/**	All product API calls → Product Service
Method	Method=GET, POST	Only allow GET and POST
Header	Header=X-Request-Id, \d+	Route only requests with numeric ID
Query	Query=version, v2	Route v2 API calls to new service version
Weight	Weight=group1, 8	Canary release: 80% stable, 20% new (Session 14)

 Forward reference: the Weight predicate returns in Session 14 for Canary deployments.

Microservices Course · Session 2

Filters — GatewayFilter vs GlobalFilter

GatewayFilter (Local)

- Applied to ONE specific route
- Defined inside the route config
- Example: StripPrefix, RewritePath
- Example: AddRequestHeader for one service

GlobalFilter

- Applied to ALL routes
- Defined as a separate @Component
- Example: Logging, RequestId, CORS
- Example: JWT validation (Session 3)

Filters We Use Today



StripPrefix=0

Keeps the full path when forwarding — used on our product route



AddResponseHeader

Adds X-Platform: microservices-pro to every response



AddRequestHeader

Injects headers before forwarding (e.g. X-Request-Source)



RewritePath

Transforms the path before forwarding (e.g. /old → /api/v1)

Creating the Gateway Service

Spring Initializr — New Project

Group `com.microservices.pro`

Artifact `api-gateway`

Dependencies `Gateway, Eureka Discovery Client, Spring Boot Actuator`



Critical: Do NOT add Spring Web dependency.
Gateway uses WebFlux (reactive) — adding Web creates a startup conflict.

Configuring Routes in application.yml

application.yml – API Gateway

```
server:
  port: 8080
spring:
  cloud:
    gateway:
      routes:
        - id: product-service
          uri: lb://PRODUCT-SERVICE    # lb = load balanced via Eureka
          predicates:
            - Path=/api/products/**
          filters:
            - StripPrefix=0
            - AddResponseHeader=X-Platform, microservices-pro
eureka:
  client:
    service-url:
      defaultZone: http://localhost:8761/eureka/
```

Adding a Global Logging Filter

`LoggingFilter.java`

```
@Component
public class LoggingFilter implements GlobalFilter, Ordered {

    @Override
    public Mono<Void> filter(ServerWebExchange exchange,
                           GatewayFilterChain chain) {
        log.info("[GATEWAY] {} {}",
            exchange.getRequest().getMethod(),
            exchange.getRequest().getURI().getPath());

        return chain.filter(exchange);
    }

    @Override
    public int getOrder() {
        return Ordered.HIGHEST_PRECEDENCE; // runs first
    }
}
```

Test the Gateway + Show Load Balancing

TEST 1 — Basic Routing

```
curl http://localhost:8080/api/products
```

→ *returns product list, proxied transparently from Product Service*

TEST 2 — Verify Logging Filter

```
[GATEWAY] GET /api/products
```

→ *Gateway console shows this line for every request*

TEST 3 — Load Balancing in Action

Start a 2nd Product Service instance on :8082. Call the Gateway 4 times — watch logs alternate 8081 / 8082 / 8081 / 8082.

Lab 2A — API Gateway Setup & Product Route

1

Create & Configure the Gateway

New service, port 8080, route to Product Service via Eureka

2

Implement the Logging GlobalFilter

Log method + path on every request through the Gateway

3

Write Unit Tests

Minimum 2 tests for the filter behaviour

Lab 2A Acceptance Criteria

- ✓ Gateway starts on port 8080 without errors
- ✓ API-GATEWAY appears in Eureka dashboard alongside PRODUCT-SERVICE
- ✓ GET localhost:8080/api/products returns product list (proxied)
- ✓ Gateway console shows "[GATEWAY] GET /api/products" log line
- ✓ Response includes X-Platform: microservices-pro header
- ✓ All unit tests pass: mvn test
- ✓ Commit pushed: session-02: add-api-gateway-with-product-route

What We Added to the Platform Today



Capability Added

Unified Single Entry Point & Request Routing

All external traffic enters through one address. Clients never call services directly.



Services Added

NEW: API Gateway (8080)

- Eureka integration + product route
- Global Logging Filter
- X-Platform response header

- ? Session 1: Config + Eureka + Product Service + Docker Compose
- ? Session 2: API Gateway — Single Entry Point
- ? Session 3: Auth Filter + Rate Limiting

Config



Eureka



Product



Gateway



Order



Payment



Inventory



Notif.





Daily Quiz

8 Questions · 10 Minutes

Topics: API Gateway Purpose · Route Predicates · GlobalFilter vs GatewayFilter · Load Balancing

Checkpoint Commit & Homework

Checkpoint — Definition of Done

- ☐ Gateway routes /api/products correctly
- ☐ API-GATEWAY visible in Eureka
- ☐ LoggingFilter shows requests in console
- ☐ All unit tests pass
- ☐ Pushed: session-02: ...

Pre-Session 3 Reading (15 min)

JWT Authentication + Rate Limiting Basics
spring.io/projects/spring-security

Knowledge Check Questions for Session 3:

- What is a JWT token made of?
- Where should authentication happen — Gateway or service?
- What is a token bucket rate limiter?

Common Issues & Solutions



Startup error: WebFlux/Servlet conflict

Spring Web dependency was added by mistake — remove it, Gateway needs only WebFlux



503 Service Unavailable from Gateway

Check Product Service is registered in Eureka before testing the route



Route not matching the request

Verify Path predicate pattern — /** is required for wildcard sub-paths



No log lines from LoggingFilter

Confirm @Component is present and getOrder() returns a valid precedence



X-Platform header missing in response

AddResponseHeader filter must be under filters:, not predicates:



Session 2 Complete

The platform now has a single entry point: the API Gateway

Next: Session 3 — API Gateway Advanced (JWT + Rate Limiting) · Wednesday, 3:00–5:30 PM, Online

[microservices-pro-course](#) · [microservices-pro-platform](#) · Dr. Sayed Baladoh